Álgebra Lineal

Conceptualización - Parte 2

Sergio A. Cantillo

sacantillo@uao.edu.co

Facultad de Ingeniería y Ciencias Básicas
Universidad Autónoma de Occidente





Agenda

1. Numpy - Manejo y Extracción de datos en Arrays

Agenda

1. Numpy - Manejo y Extracción de datos en Arrays

2. Matplotlib - Desarrollo de gráficos
   - Flujo de Trabajo
   - Subplots
   - Tipos de gráfico
   - Ajustes de estilo de gráficos

Agenda

1. Numpy - Manejo y Extracción de datos en Arrays

2. Matplotlib - Desarrollo de gráficos
   - Flujo de Trabajo
   - Subplots
   - Tipos de gráfico
   - Ajustes de estilo de gráficos

3. Otros conceptos de Algebra Lineal
   - Transformación Lineal
   - Autovalores y Autovectores

Agenda

1. Numpy - Manejo y Extracción de datos en Arrays

2. Matplotlib - Desarrollo de gráficos
   - Flujo de Trabajo
   - Subplots
   - Tipos de gráfico
   - Ajustes de estilo de gráficos

3. Otros conceptos de Algebra Lineal
   - Transformación Lineal
   - Autovalores y Autovectores

4. SVD
   - Aplicaciones

Agenda

1. Numpy - Manejo y Extracción de datos en Arrays
2. Matplotlib - Desarrollo de gráficos
   - Flujo de Trabajo
   - Subplots
   - Tipos de gráfico
   - Ajustes de estilo de gráficos
3. Otros conceptos de Algebra Lineal
   - Transformación Lineal
   - Autovalores y Autovectores
4. SVD
   - Aplicaciones
5. Referencias

Manipulación de Arrays

Array Manipulation

Transposing Array

```
>>> i = np.transpose(b)
```
Permute array dimensions

```
>>> i.T
```
Permute array dimensions

Changing Array Shape

```
>>> b.ravel()
```
Flatten the array

```
>>> g.reshape(3,-2)
```
Reshape, but don't change data

Adding/Removing Elements

```
>>> h.resize((2,6))
```
Return a new array with shape (2,6)

```
>>> np.append(h,g)
```
Append items to an array

```
>>> np.insert(a, 1, 5)
```
Insert items in an array

```
>>> np.delete(a,[1])
```
Delete items from an array

Combining Arrays

```
>>> np.concatenate((a,d),axis=0)
 array([ 1,  2,  3, 10, 15, 20])
```
Concatenate arrays

```
>>> np.vstack((a,b))
 array([[ 1. ,  2. ,  3. ],
        [ 1.5,  2. ,  3. ],
        [ 4. ,  5. ,  6. ]])
```
Stack arrays vertically (row-wise)

```
>>> np.r_[e,f]
```
Stack arrays vertically (row-wise)

```
>>> np.hstack((e,f))
 array([[ 7.,  7.,  1.,  0.],
        [ 7.,  7.,  0.,  1.]])
```
Stack arrays horizontally (column-wise)

```
>>> np.column_stack((a,d))
 array([[ 1, 10],
        [ 2, 15],
        [ 3, 20]])
```
Create stacked column-wise arrays

```
>>> np.c_[a,d]
```
Create stacked column-wise arrays

Splitting Arrays

```
>>> np.hsplit(a,3)
 [array([1]),array([2]),array([3])]
```
Split the array horizontally at the 3rd index

```
>>> np.vsplit(c,2)
[array([[[ 1.5,  2. ,  1. ],
        [ 4. ,  5. ,  6. ]]]),
 array([[[ 3.,  2.,  3.],
         [ 4.,  5.,  6.]]])]
```
Split the array vertically at the 2nd index

A tener en cuenta!

- Todos estos métodos reciben *arrays* y devuelven *arrays*.
- los valores negativos en los parametros de las funciones indican desinteres en ellos (inferencia automática).

Seccionado - Selección de Arrays

Subsetting, Slicing, Indexing

Also see Lists

Subsetting

```
>>> a[2]
3
```


Select the element at the 2nd index

```
>>> b[1,2]
6.0
```


Select the element at row 1 column 2 (equivalent to `b[1][2]`)

Slicing

```
>>> a[0:2]
array([1, 2])
```


Select items at index 0 and 1

```
>>> b[0:2,1]
array([ 2.,  5.])
```


Select items at rows 0 and 1 in column 1

```
>>> b[:1]
array([[1.5, 2., 3.]])
```


Select all items at row 0 (equivalent to `b[0:1, :]`)

```
>>> c[1,...]
array([[[ 3.,  2.,  1.],
        [ 4.,  5.,  6.]]])
```

Same as `[1,:,:]`

```
>>> a[ : :-1]
array([3, 2, 1])
```

Reversed array `a`

Boolean Indexing

```
>>> a[a<2]
array([1])
```


Select elements from `a` less than 2

Fancy Indexing

```
>>> b[[1, 0, 1, 0],[0, 1, 2, 0]]
array([ 4. , 2. , 6. , 1.5])
```

Select elements (1,0), (0,1), (1,2) and (0,0)

```
>>> b[[1, 0, 1, 0]][:,[0,1,2,0]]
array([[ 4. ,5. , 6. , 4. ],
       [ 1.5, 2. , 3. , 1.5],
       [ 4. , 5. , 6. , 4. ],
       [ 1.5, 2. , 3. , 1.5]])
```

Select a subset of the matrix's rows and columns

A tener en cuenta!

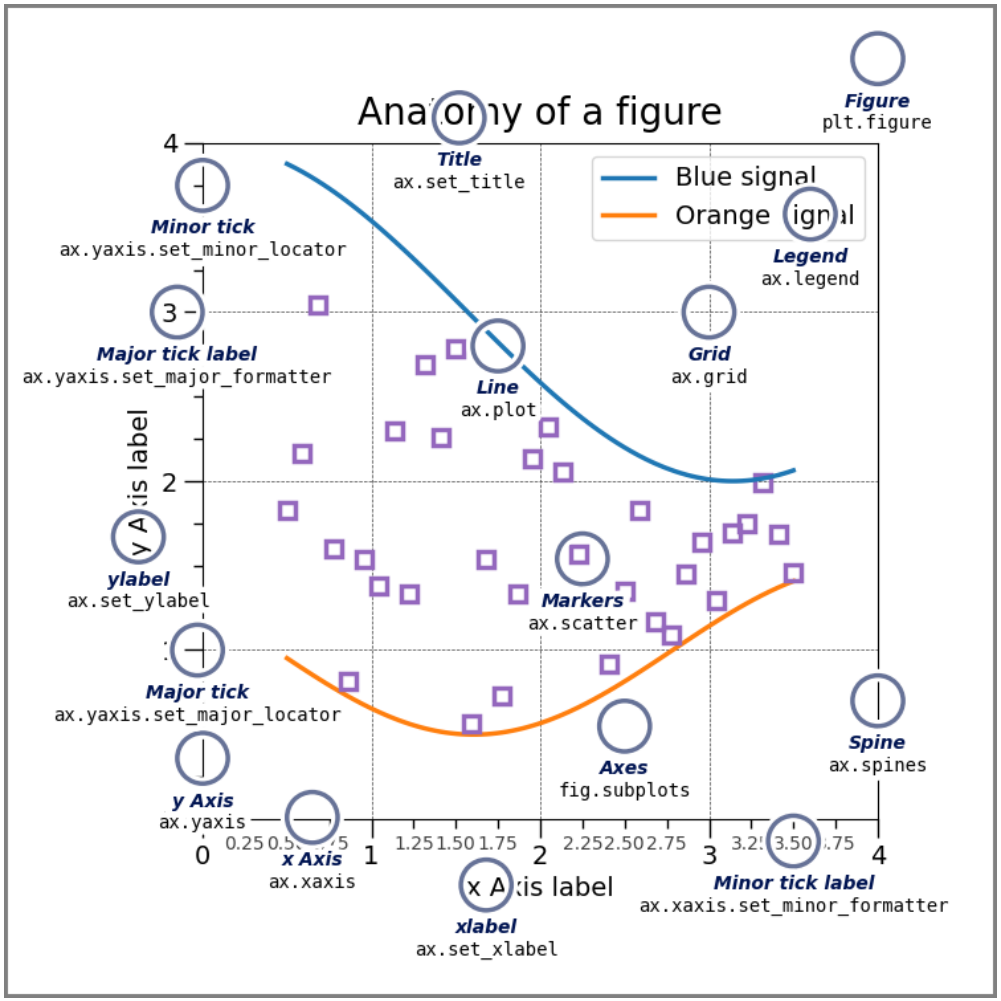
- La selección siempre cuenta desde cero como primer valor.
- Las coordenadas cuando el array presenta mas de una dimensión, se indican con coma.
- Se puede expresar una condición entre corchetes para filtrar información.

Librería para Gráficos: Matplotlib

- Es el paquete Python más utilizado en el ámbito de la ciencia de datos para representaciones gráficas.
- La forma más común de utilizar esta librería es usar el alias **plt**.
- En su anatomía, se incluyen ejes (Axis), marcas (Ticks), leyendas (Legend)

Importación

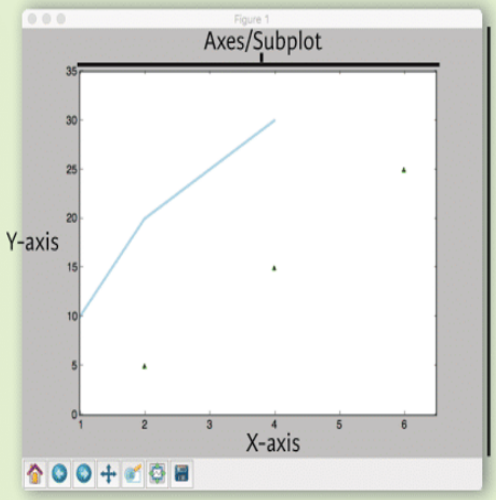
```
import matplotlib.pyplot as plt
```



Matplotlib: Flujo de trabajo

Plot Anatomy & Workflow

Plot Anatomy



Workflow

The basic steps to creating plots with matplotlib are:

1 Prepare data 2 Create plot 3 Plot 4 Customize plot 5 Save plot 6 Show plot

```
>>> import matplotlib.pyplot as plt
>>> x = [1,2,3,4]                    Step 1
>>> y = [10,20,25,30]
>>> fig = plt.figure()               Step 2
>>> ax = fig.add_subplot(111)        Step 3
>>> ax.plot(x, y, color='lightblue', linewidth=3)   Step 3, 4
>>> ax.scatter([2,4,6],
               [5,15,25],
               color='darkgreen',
               marker='^')
>>> ax.set_xlim(1, 6.5)
>>> plt.savefig('foo.png')
>>> plt.show()                       Step 6
```

Matplotlib: Preparación de datos

1 Prepare The Data

Also see Lists & NumPy

1D Data

```
>>> import numpy as np
>>> x = np.linspace(0, 10, 100)
>>> y = np.cos(x)
>>> z = np.sin(x)
```

2D Data or Images

```
>>> data = 2 * np.random.random((10, 10))
>>> data2 = 3 * np.random.random((10, 10))
>>> Y, X = np.mgrid[-3:3:100j, -3:3:100j]
>>> U = -1 - X**2 + Y
>>> V = 1 + X - Y**2
>>> from matplotlib.cbook import get_sample_data
>>> img = np.load(get_sample_data('axes_grid/bivariate_normal.npy'))
```

- Esta librería admite *arrays* (1D, 2D, 3D), archivos de NumPy (.npy), así como *Listas* para su graficación.
- Por temas de orden en la graficación, se esperan variables separadas y bien definidas.

Matplotlib: Creación del gráfico

2 Create Plot

```
>>> import matplotlib.pyplot as plt
```

Figure

```
>>> fig = plt.figure()
>>> fig2 = plt.figure(figsize=plt.figaspect(2.0))
```

Axes

All plotting is done with respect to an `Axes`. In most cases, a subplot will fit your needs. A subplot is an axes on a grid system.

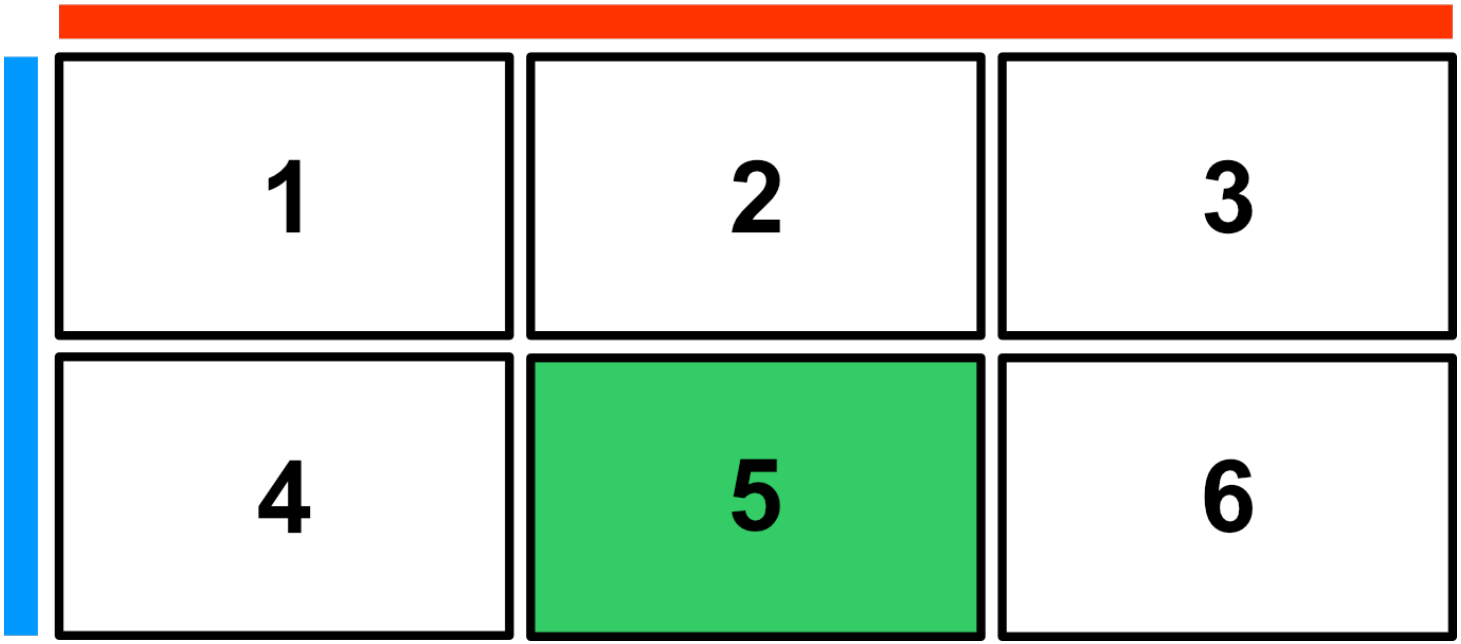
```
>>> fig.add_axes()
>>> ax1 = fig.add_subplot(221) # row-col-num
>>> ax3 = fig.add_subplot(212)
>>> fig3, axes = plt.subplots(nrows=2,ncols=2)
>>> fig4, axes2 = plt.subplots(ncols=3)
```

- Se realiza a traves de la función **figure**.
- Es completamente modificable, incluso permite la creación de *subgráficos* (Subplots - plt.subplots()).
- Es posible el uso de *Axes* como forma personalizada de graficación.

Matplotlib: Creación del gráfico

Subplots

```
fig.add_subplot(2, 3, 5)
```



- Forma de representación de varios gráficos en forma simultánea.
- Se ubican en formato de filas y columnas, ademas de numeración de gráficos de izquierda a derecha y de arriba a abajo.
- Gran utilidad en análisis de multiples dimensiones de un conjunto de datos.

Matplotlib: Graficación

3 Plotting Routines

1D Data

`>>> lines = ax.plot(x,y)`	Draw points with lines or markers connecting them
`>>> ax.scatter(x,y)`	Draw unconnected points, scaled or colored
`>>> axes[0,0].bar([1,2,3],[3,4,5])`	Plot vertical rectangles (constant width)
`>>> axes[1,0].barh([0.5,1,2.5],[0,1,2])`	Plot horiontal rectangles (constant height)
`>>> axes[1,1].axhline(0.45)`	Draw a horizontal line across axes
`>>> axes[0,1].axvline(0.65)`	Draw a vertical line across axes
`>>> ax.fill(x,y,color='blue')`	Draw filled polygons
`>>> ax.fill_between(x,y,color='yellow')`	Fill between y-values and o

Vector Fields

`>>> axes[0,1].arrow(0,0,0.5,0.5)`	Add an arrow to the axes
`>>> axes[1,1].quiver(y,z)`	Plot a 2D field of arrows
`>>> axes[0,1].streamplot(X,Y,U,V)`	Plot 2D vector fields

Data Distributions

`>>> ax1.hist(y)`	Plot a histogram
`>>> ax3.boxplot(y)`	Make a box and whisker plot
`>>> ax3.violinplot(z)`	Make a violin plot

2D Data or Images

```
>>> fig, ax = plt.subplots()
>>> im = ax.imshow(img,
                   cmap='gist_earth',
                   interpolation='nearest',
                   vmin=-2,
                   vmax=2)
```

Colormapped or RGB arrays

`>>> axes2[0].pcolor(data2)`	Pseudocolor plot of 2D array
`>>> axes2[0].pcolormesh(data)`	Pseudocolor plot of 2D array
`>>> CS = plt.contour(Y,X,U)`	Plot contours
`>>> axes2[2].contourf(data1)`	Plot filled contours
`>>> axes2[2]= ax.clabel(CS)`	Label a contour plot

- Despues de seleccionar la figura y el formato de presentación de grafico, se selecciona que tipo de gráfico se requiere.
- Se grafica en formato (x,y) para elementos 2D.

Matplotlib: Ajustes estéticos

4 Customize Plot

Colors, Color Bars & Color Maps

```
>>> plt.plot(x, x, x, x**2, x, x**3)
>>> ax.plot(x, y, alpha = 0.4)
>>> ax.plot(x, y, c='k')
>>> fig.colorbar(im, orientation='horizontal')
>>> im = ax.imshow(img,
                   cmap='seismic')
```

Markers

```
>>> fig, ax = plt.subplots()
>>> ax.scatter(x,y,marker=".")
>>> ax.plot(x,y,marker="o")
```

Linestyles

```
>>> plt.plot(x,y,linewidth=4.0)
>>> plt.plot(x,y,ls='solid')
>>> plt.plot(x,y,ls='--')
>>> plt.plot(x,y,'--',x**2,y**2,'-.')
>>> plt.setp(lines,color='r',linewidth=4.0)
```

Text & Annotations

```
>>> ax.text(1,
            -2.1,
            'Example Graph',
            style='italic')
>>> ax.annotate("Sine",
                xy=(8, 0),
                xycoords='data',
                xytext=(10.5, 0),
                textcoords='data',
                arrowprops=dict(arrowstyle="->",
                                connectionstyle="arc3"),)
```

Mathtext

```
>>> plt.title(r'$sigma_i=15$', fontsize=20)
```

Limits, Legends & Layouts

Limits & Autoscaling

```
>>> ax.margins(x=0.0,y=0.1)
```
Add padding to a plot

```
>>> ax.axis('equal')
```
Set the aspect ratio of the plot to 1

```
>>> ax.set(xlim=[0,10.5],ylim=[-1.5,1.5])
```
Set limits for x-and y-axis

```
>>> ax.set_xlim(0,10.5)
```
Set limits for x-axis

Legends

```
>>> ax.set(title='An Example Axes',
           ylabel='Y-Axis',
           xlabel='X-Axis')
```
Set a title and x-and y-axis labels

```
>>> ax.legend(loc='best')
```
No overlapping plot elements

Ticks

```
>>> ax.xaxis.set(ticks=range(1,5),
                 ticklabels=[3,100,-12,"foo"])
```
Manually set x-ticks

```
>>> ax.tick_params(axis='y',
                   direction='inout',
                   length=10)
```
Make y-ticks longer and go in and out

Subplot Spacing

```
>>> fig3.subplots_adjust(wspace=0.5,
                         hspace=0.3,
                         left=0.125,
                         right=0.9,
                         top=0.9,
                         bottom=0.1)
```
Adjust the spacing between subplots

```
>>> fig.tight_layout()
```
Fit subplot(s) in to the figure area

Axis Spines

```
>>> ax1.spines['top'].set_visible(False)
```
Make the top axis line for a plot invisible

```
>>> ax1.spines['bottom'].set_position(('outward',10))
```
Move the bottom axis line outward

Matplotlib: Presentación/Guardado de Gráficos

5 Save Plot

Save figures

```
>>> plt.savefig('foo.png')
```

Save transparent figures

```
>>> plt.savefig('foo.png', transparent=True)
```

6 Show Plot

```
>>> plt.show()
```

Close & Clear

`>>> plt.cla()`	Clear an axis
`>>> plt.clf()`	Clear the entire figure
`>>> plt.close()`	Close a window

- Se pueden guardar los graficos hechos en esta librería en distintos formatos de imagen (.png, .jpg, .bmp, .tiff, .raw, entre otros).
- Sin el comando **show()**, no se muestra nada al usuario (Python nativo - archivos .py) ¡Ojo!.

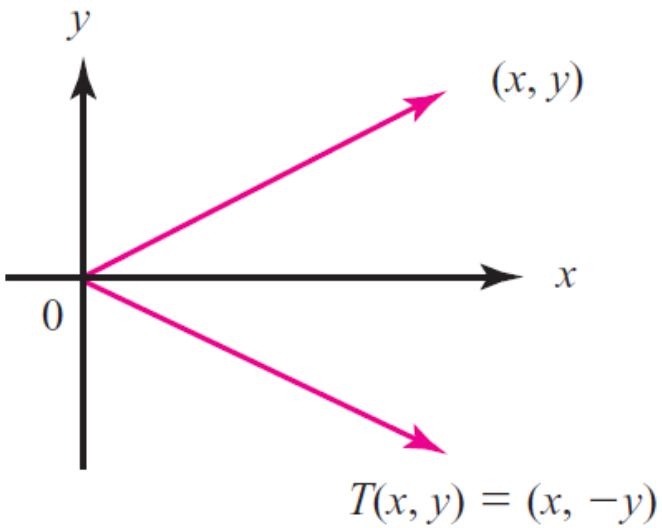
Otros Conceptos de Algebra Lineal

Transformación Lineal

Definición

Sean V y W dos espacios vectoriales, una **transformación lineal** es una *función* que asigna a cada vector $v \in V$ un vector único $Tv \in W$ y que satisface de suma de vectores y multiplicación por un escalar. Se asocia a la idea de movimiento.

$$T(u+v) = Tu + Tv$$
$$T(\alpha v) = \alpha Tv$$



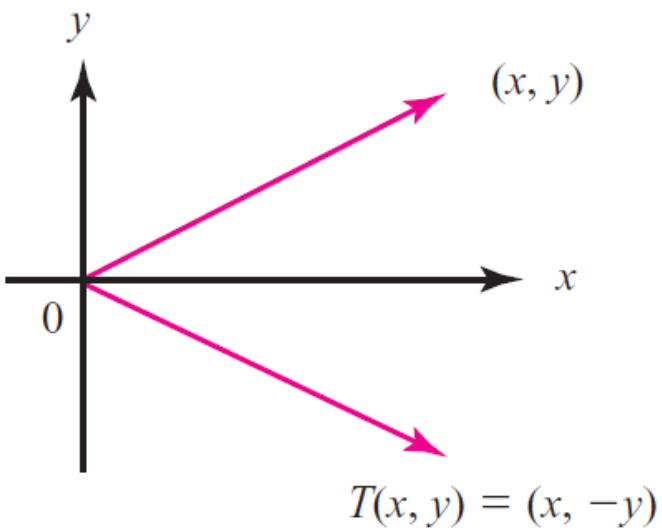
Transformación Lineal de $\mathbb{R}^2$ en $\mathbb{R}^2$: reflexión con respecto al eje x.

$$T\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \\ -y \end{bmatrix}$$

Transformación Lineal

Condiciones

- Las líneas deben permanecer rectas (no curvan)
- El origen se mantiene fijo (no hay desplazamiento)

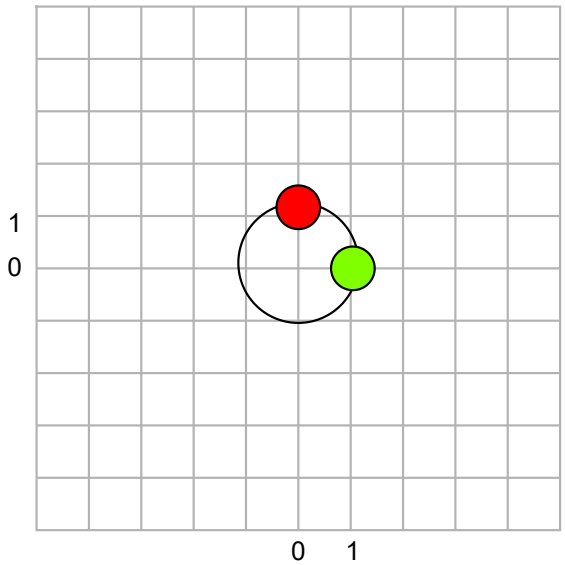


Transformación Lineal de $\mathbb{R}^2$ en $\mathbb{R}^2$: reflexión con respecto al eje x.

$$T\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} x \\ -y \end{bmatrix}$$

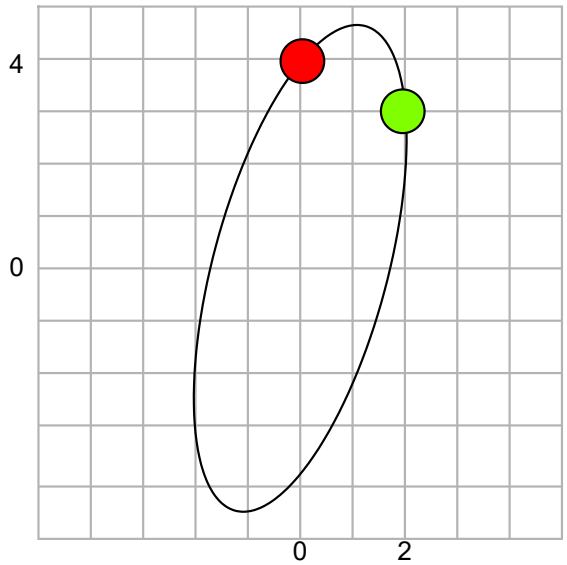
Transformación Lineal

Transformación Lineal = Operaciones Matriciales



$$M = \begin{bmatrix} 2 & 0 \\ 3 & 4 \end{bmatrix}$$

(p, q)	$(2p + 0q, 3p + 4q)$
$(1, 0)$	$(2, 3)$
$(0, 1)$	$(0, 4)$



Autovalores y Autovectores de una Matriz

Una transformación queda completamente determinada por sus vectores propios y valores propios.

$$Tv = \lambda v$$

Autovectores (eigenvectores) y Autovalores (eigenvalores)

El vector propio v de la matriz A de $n \times n$ es un vector diferente de cero tal que $Av = \lambda v$ donde λ es un valor escalar real que recibe el nombre de valor característico, propio o eigenvalor.

Pasos para calcular valores y vectores propios:

1. Encontrar $p(\lambda) = \det(A - \lambda I)$
2. Encontrar las raíces $\lambda_1, \lambda_2, ..., \lambda_m$ de $p(\lambda) = 0$
3. Resolver el sistema homogéneo $(A - \lambda_i I)v = 0$

https://www.youtube.com/watch?v=GxOPaWI9eYo&list=PLIb_io8a5NB2DddFf-PwvZDCOUNT1GZoA&index=14

Cálculo de Autovalores y Autovectores

Ejemplo

Consideremos la siguiente matriz A de 3×3:

$$A = \begin{bmatrix} 2 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{bmatrix}$$

Se buscan los autovalores y autovectores de la matriz A.

Empezamos calculando el polinomio característico:

$$\det(A - \lambda I) = 0$$

Donde λ es el autovalor que estamos buscando y I es la matriz identidad. Esto nos lleva a la ecuación característica:

$$\begin{vmatrix} 2-\lambda & -1 & 0 \\ -1 & 3-\lambda & -1 \\ 0 & -1 & 2-\lambda \end{vmatrix} = 0$$

Cálculo de Autovalores y Autovectores

Simplificamos la determinante y resolvemos la ecuación característica:

$$(2-\lambda)\cdot[(3-\lambda)\cdot(2-\lambda)-1]-(-1)[(-1)(2-\lambda)]=0$$

$$(2-\lambda)\cdot(\lambda^2-5\lambda+5)-(2-\lambda)=0$$

$$(2-\lambda)\cdot[(\lambda^2-5\lambda+5)-1]=0$$

$$(2-\lambda)\cdot[(\lambda^2-5\lambda+4]=0$$

Esto nos da tres posibles autovalores: $\lambda_1=2$, $\lambda_2=1$, y $\lambda_3=4$.

Cálculo de Autovalores y Autovectores

Ahora que hemos encontrado los autovalores, procedemos a calcular los autovectores correspondientes a cada autovalor.

Para $\lambda = 2$, resolvemos $(A - 2I)\mathbf{v} = \mathbf{0}$ para encontrar los autovectores $\mathbf{v}_1$:

$$\begin{bmatrix} 2 & -1 & 0 \\ -1 & 3 & -1 \\ 0 & -1 & 2 \end{bmatrix} - 2 \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 1 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 1 & -1 \\ 0 & -1 & 0 \end{bmatrix} \cdot \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

Resolviendo el sistema, obtenemos el autovector $\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}$.

Cálculo de Autovalores y Autovectores

Continuando de manera similar para los otros autovalores:

Para $\lambda = 1$, resolvemos $(A - I)\mathbf{v} = \mathbf{0}$:

$$\mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

Para $\lambda = 4$, resolvemos $(A - 4I)\mathbf{v} = \mathbf{0}$:

$$\mathbf{v}_3 = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

Cálculo de Autovalores y Autovectores

```python
import numpy as np

# Definición de la matriz 3x3
A = np.array([[2, -1, 0],
              [-1, 3, -1],
              [0, -1, 2]])

eigenvalues, eigenvectors = np.linalg.eig(A)
eigenvectors /= eigenvectors[0, :]

print("Autovalores:")
print(eigenvalues)

print("Autovectores")
print(eigenvectors)
```

✓ 0.0s

```
Autovalores:
[4. 2. 1.]
Autovectores
[[ 1.00000000e+00  1.00000000e+00  1.00000000e+00]
 [-2.00000000e+00 -3.17570282e-16  1.00000000e+00]
 [ 1.00000000e+00 -1.00000000e+00  1.00000000e+00]]
```

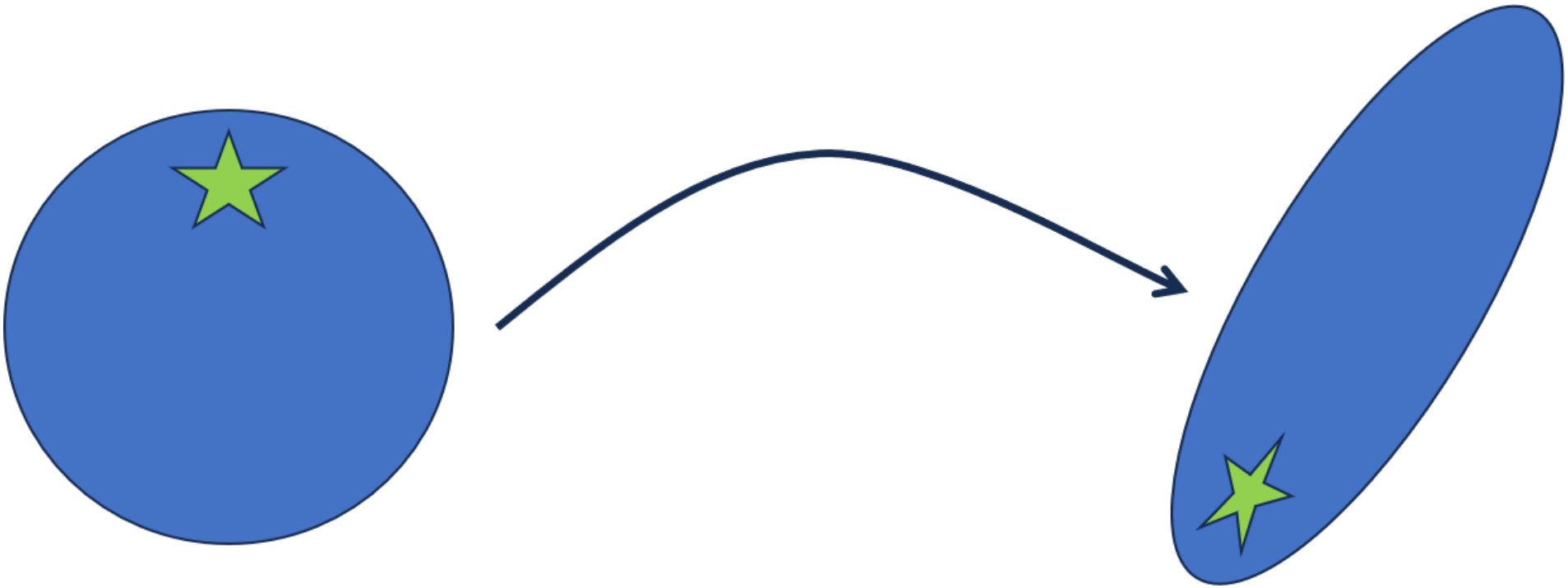
En resumen, los autovalores de la matriz A son $\lambda_1 = 2$, $\lambda_2 = 1$, y $\lambda_3 = 4$, con los siguientes autovectores respectivos:

$$\mathbf{v}_1 = \begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}, \quad \mathbf{v}_2 = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{v}_3 = \begin{bmatrix} 1 \\ -2 \\ 1 \end{bmatrix}$$

¿Qué es la Factorización?

¿Es posible la Factorización en matrices?

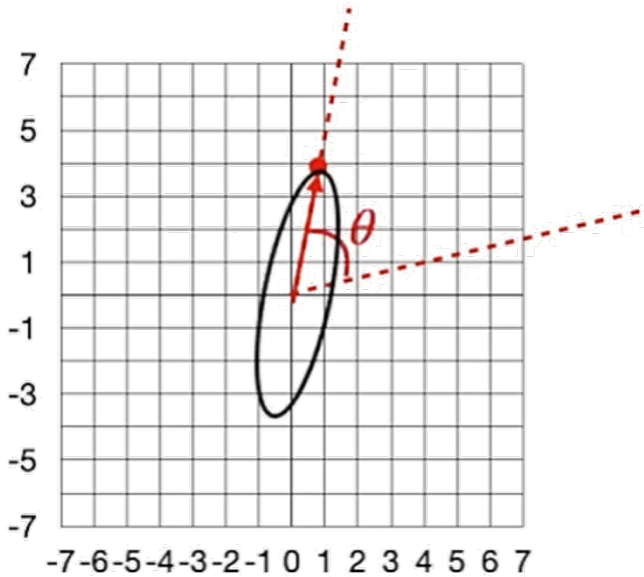
¿Como convertir?



- No se puede desplazar su centro.
- la orientacion debe ser tal cual aparece en la imagen de destino.

Matrices de Rotación



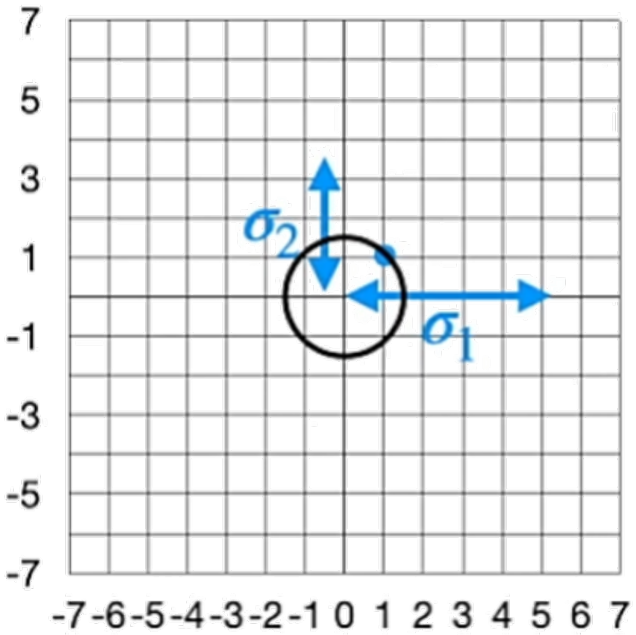




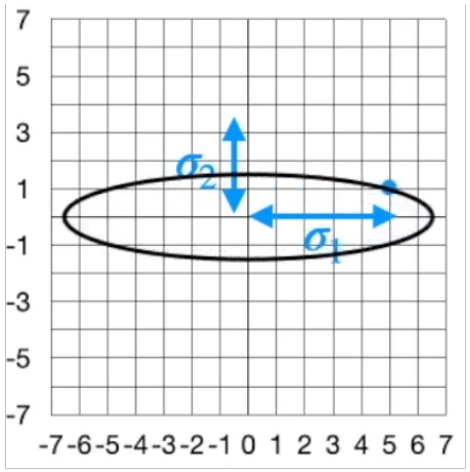


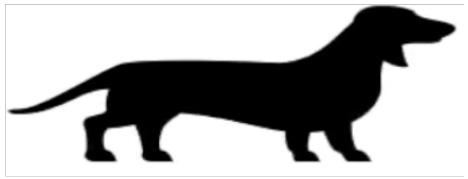
$$\begin{bmatrix} \cos\theta & -\operatorname{sen}\theta \\ \operatorname{sen}\theta & \cos\theta \end{bmatrix}$$

Matrices de Escalamiento

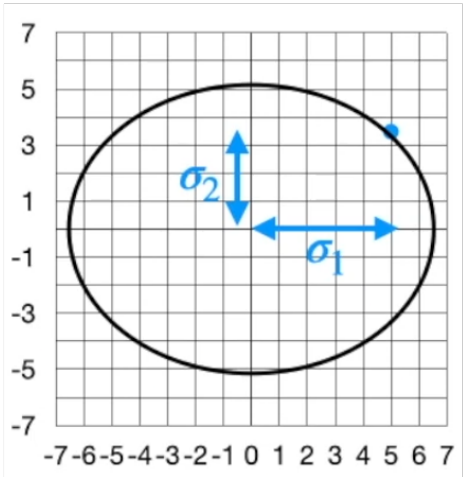


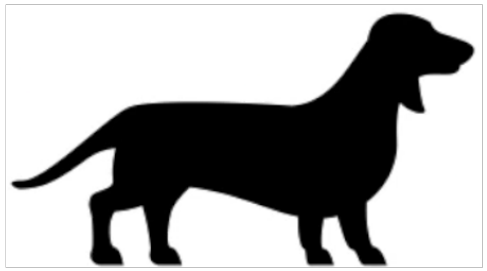






$$\begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{bmatrix}$$

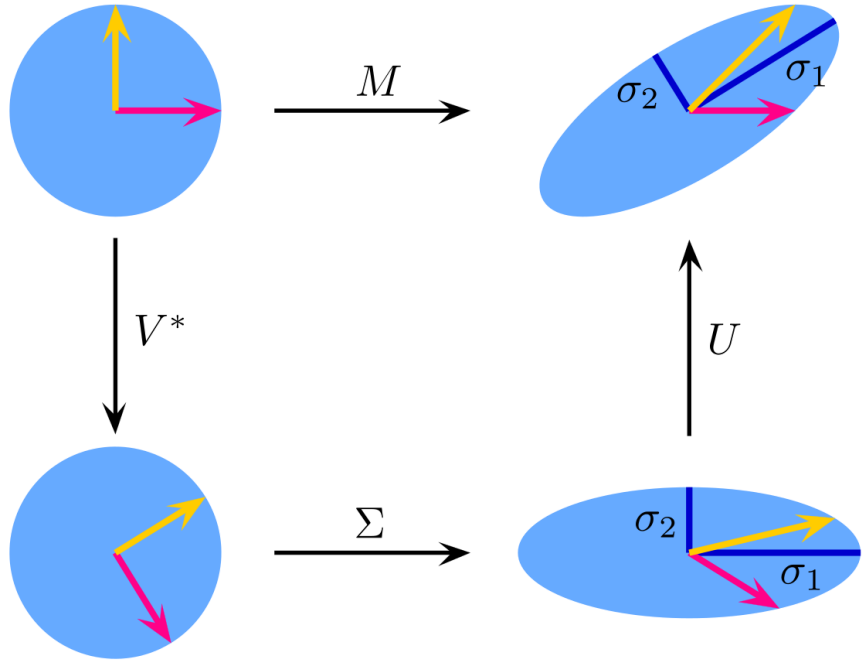




Singular Value Decomposition (SVD)

Definición

Método de descomposición de matrices. Permite la expresión de cualquier matriz en el **producto** de otras tres matrices (Escalado y Rotación)



$$M = U \cdot \Sigma \cdot V^*$$

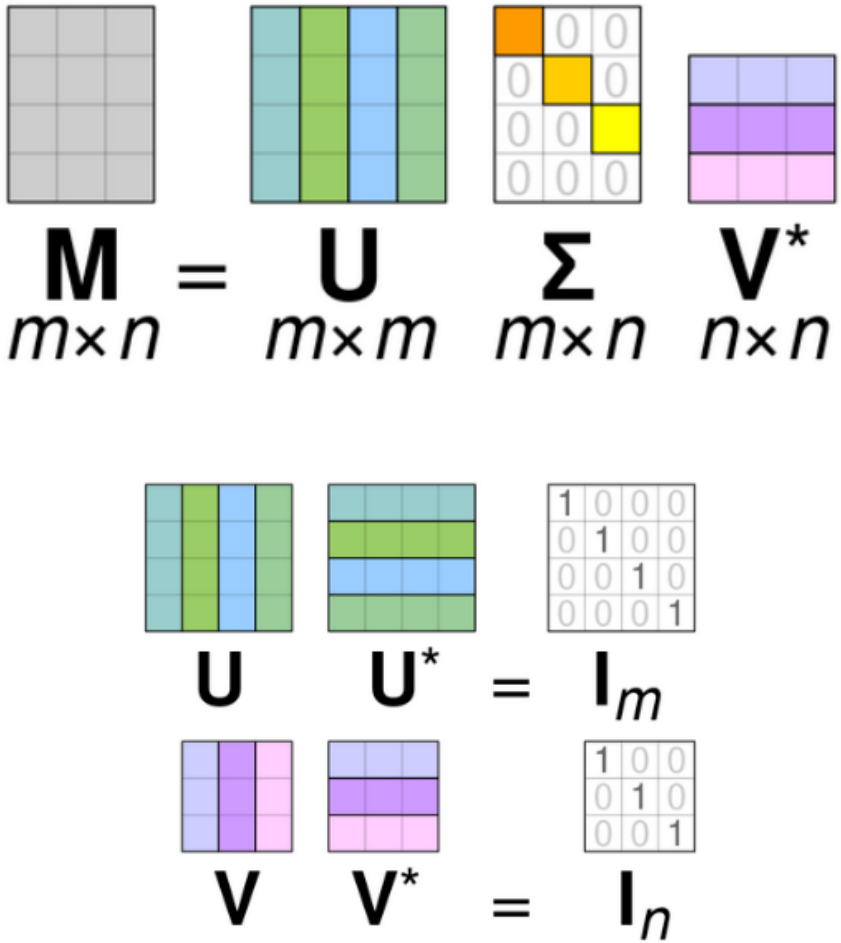
Expresión

$$M = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 \\ 0 & \sigma_2 \end{bmatrix} \begin{bmatrix} \cos\phi & \sin\phi \\ -\sin\phi & \cos\phi \end{bmatrix}$$

Singular Value Decomposition (SVD)

Condiciones

- Las columnas de U son los *autovectores* de MM^T.
- Las columnas de V son los *autovectores* de M^TM.
- $\sum$ es una matriz *diagonal* que contiene la raíz de los autovalores de MM^T (o de M^TM) ordenados en forma decreciente, cuyos elementos mayores que cero se conocen como **valores singulares**.
- U y V son matrices cuadradas *unitarias*, es decir que al multiplicarse por su transpuesta se obtiene la matriz identidad.



Singular Value Decomposition (SVD)

Ejemplo

Dada la siguiente matriz:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

Hallar su descomposición de valores singulares (SVD).

$$A^T A = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} = \begin{bmatrix} 17 & 22 & 27 \\ 22 & 29 & 36 \\ 27 & 36 & 45 \end{bmatrix}$$

$$AA^T = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix} = \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix}$$

Singular Value Decomposition (SVD)

Calculamos los autovalores de la matriz resultante $A^T A$ y su raiz serán los valores singulares:

$$\det(A^T A - \lambda I) = 0$$

$$A^T A - \lambda I = \begin{bmatrix} 17 & 22 & 27 \\ 22 & 29 & 36 \\ 27 & 36 & 45 \end{bmatrix} - \begin{bmatrix} \lambda & 0 & 0 \\ 0 & \lambda & 0 \\ 0 & 0 & \lambda \end{bmatrix} = \begin{bmatrix} 17-\lambda & 22 & 27 \\ 22 & 29-\lambda & 36 \\ 27 & 36 & 45-\lambda \end{bmatrix}$$

$$\det(A^T A - \lambda I) = \lambda^3 + 91\lambda^2 - 54\lambda = 0$$

$$\lambda_3 = 0,\ \lambda_2 = 0{,}597,\ \text{y}\ \lambda_{=}90{,}4$$

$$\sigma_1 = \sqrt{90{,}4} = \mathbf{9.508}$$

$$\sigma_2 = \sqrt{0{,}597} = \mathbf{0.772}$$

$$\sigma_3 = \sqrt{0} = \mathbf{0}$$

Singular Value Decomposition (SVD)

Calculamos los autovalores de la matriz resultante AA^T y su raiz serán los valores singulares:

$$\det(AA^T - \lambda I) = 0$$

$$AA^T - \lambda I = \begin{bmatrix} 14 & 32 \\ 32 & 77 \end{bmatrix} - \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} = \begin{bmatrix} 14-\lambda & 32 \\ 32 & 77-\lambda \end{bmatrix}$$

$$\det(AA^T - \lambda I) = \lambda^2 - 91\lambda^2 + 54\lambda = 0$$

$$\lambda_1 = 90{,}4, \text{ y } \lambda_2 = 0{,}597$$

$$\sigma_1 = \sqrt{90{,}4} = \mathbf{9.508}$$

$$\sigma_2 = \sqrt{0{,}597} = \mathbf{0.772}$$

Singular Value Decomposition (SVD)

```
import numpy as np
from numpy.linalg import svd
```
✓ 0.3s

```
A = np.array([[1,2,3],[4,5,6]])
U, Sigma, V_trans = np.linalg.svd(A)
```
✓ 0.0s

```
print(U)
```
✓ 0.0s

```
[[-0.3863177   0.92236578]
 [-0.92236578 -0.3863177 ]]
```

```
print(Sigma)
```
✓ 0.0s

```
[9.508032   0.77286964]
```

```
print(V_trans)
```
✓ 0.0s

```
[[-0.42866713 -0.56630692 -0.7039467 ]
 [-0.80596391 -0.11238241  0.58119908]
 [ 0.40824829 -0.81649658  0.40824829]]
```

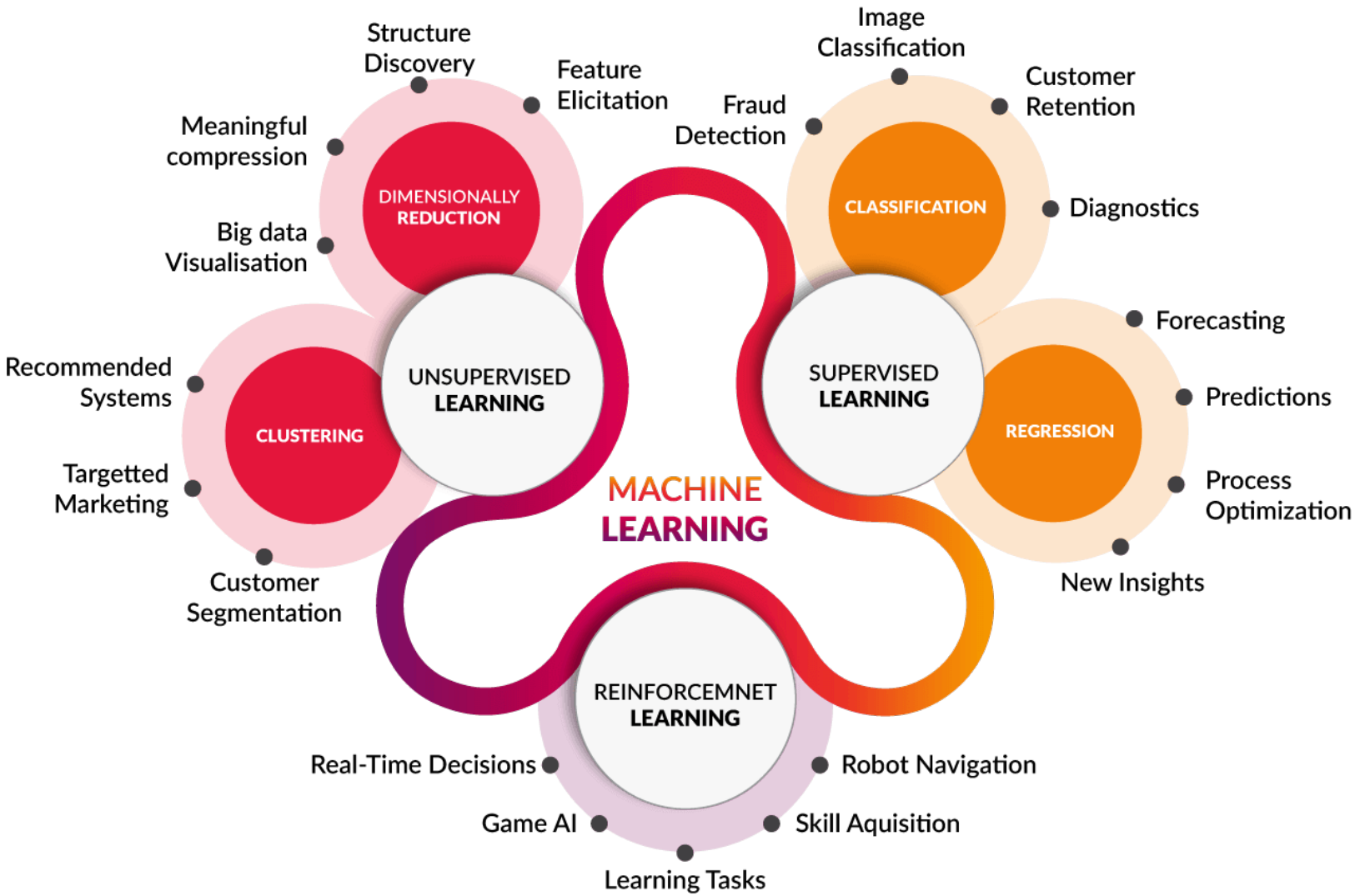
Es importante destacar que el cálculo realizado, sólo obtiene los valores que se ubican en la matriz de escalamiento σ.

$$\Sigma = \begin{bmatrix} 9{,}508 & 0 & 0 \\ 0 & 0{,}772 & 0 \end{bmatrix}$$

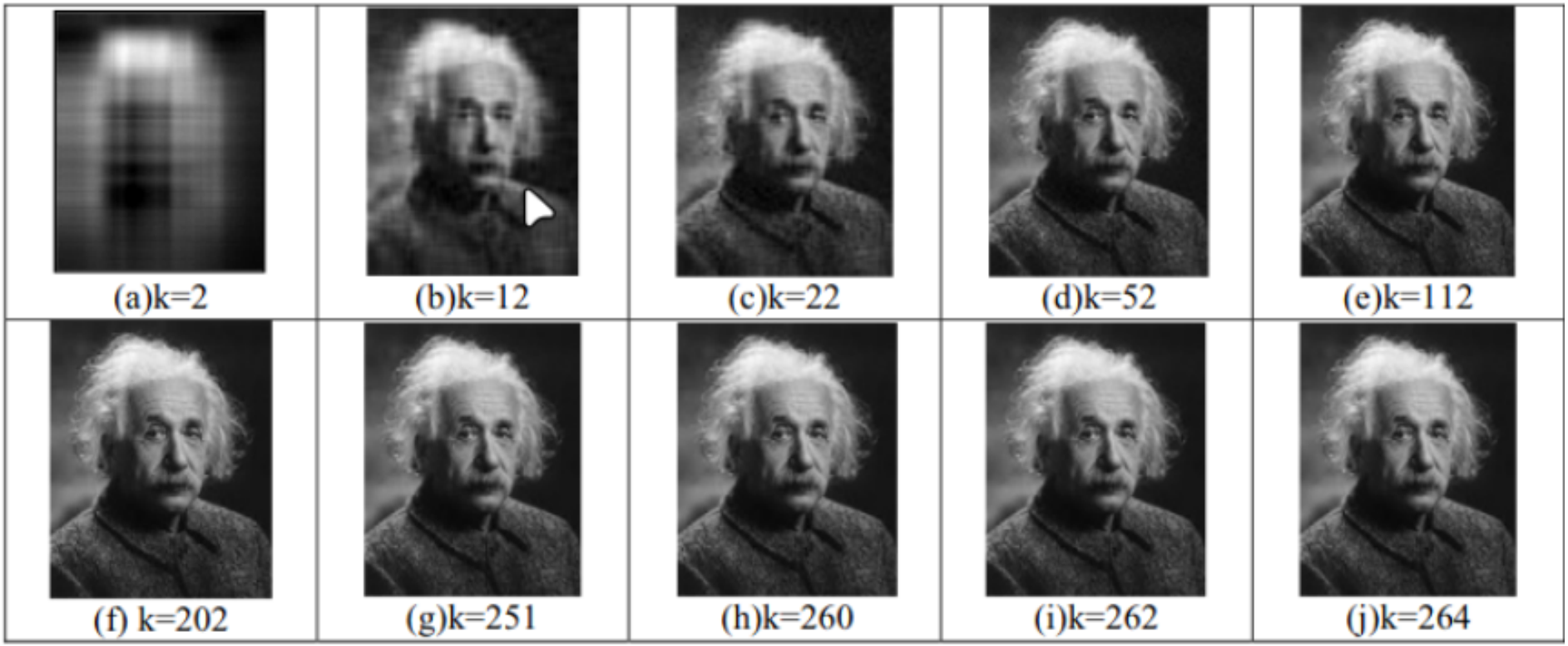
En este caso la matriz de valores singulares σ, a pesar de tener 3 autovalores es de rango 2, dado que uno de los autovalores fue 0 y se puede expresar como una matriz de 2 x 3 en este caso. Para hallar las matrices de rotación U y V, es necesario seguir el proceso iniciado con los autovalores encontrando los **autovectores** para cada caso.

Aplicaciones SVD

Contexto en Apps ML



Compresión de Imágenes

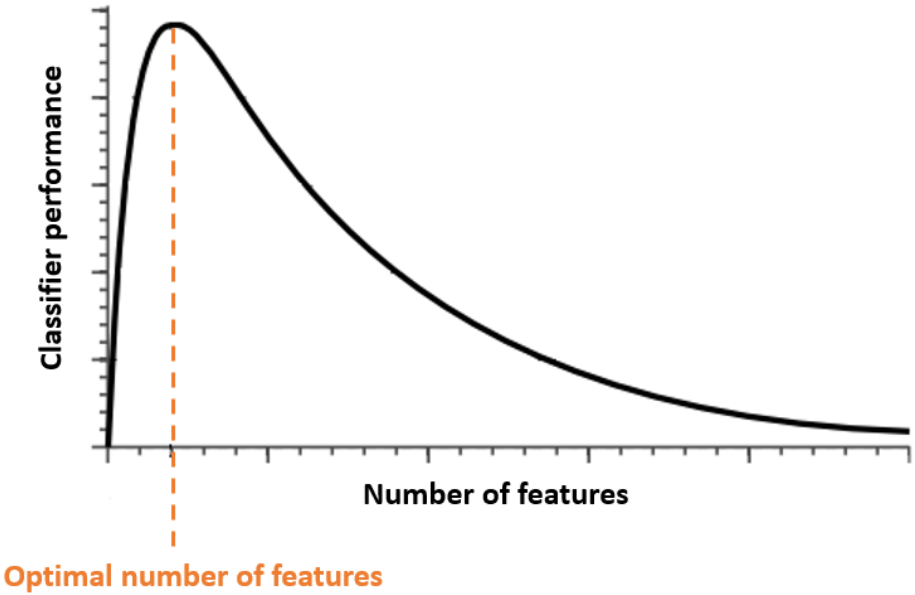


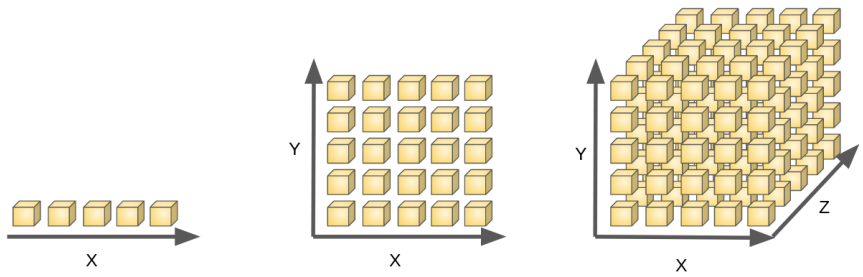
Si consideramos a la imagen como una matriz de píxeles, podemos aplicar SVD para quedarnos con los r valores singulares más grandes (aproximación de rango r), mientras más chico r mayor será la compresión.

`http://timbaumann.info/svd-image-compression-demo/`

Reducción de Dimensionalidad

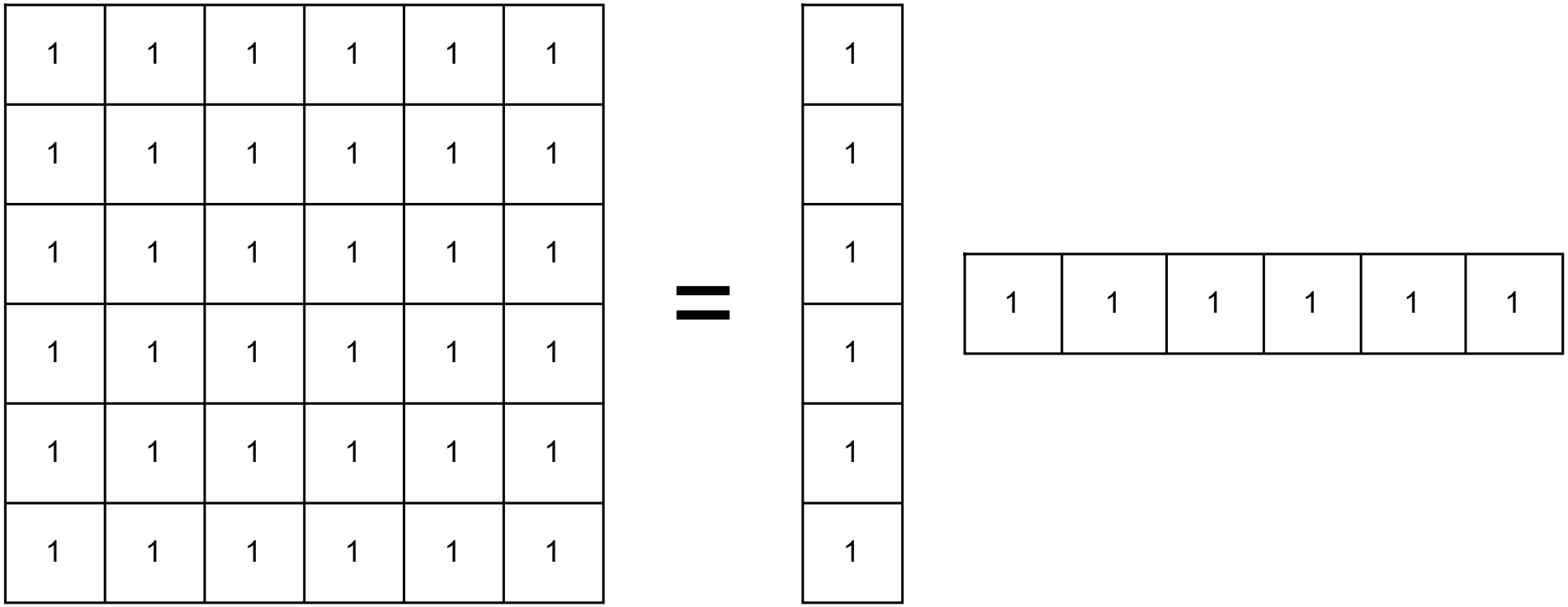
Maldición de la dimensionalidad





En ML, el número de características corresponde al numero de dimensiones del espacio en el que se representan los datos. Cuando la dimensionalidad aumenta, el espacio aumenta y los datos se vuelven más dispersos (es decir, menos oportunidades para la detección de patrones).

Reducción de Dimensionalidad I



Reducción de Dimensionalidad II

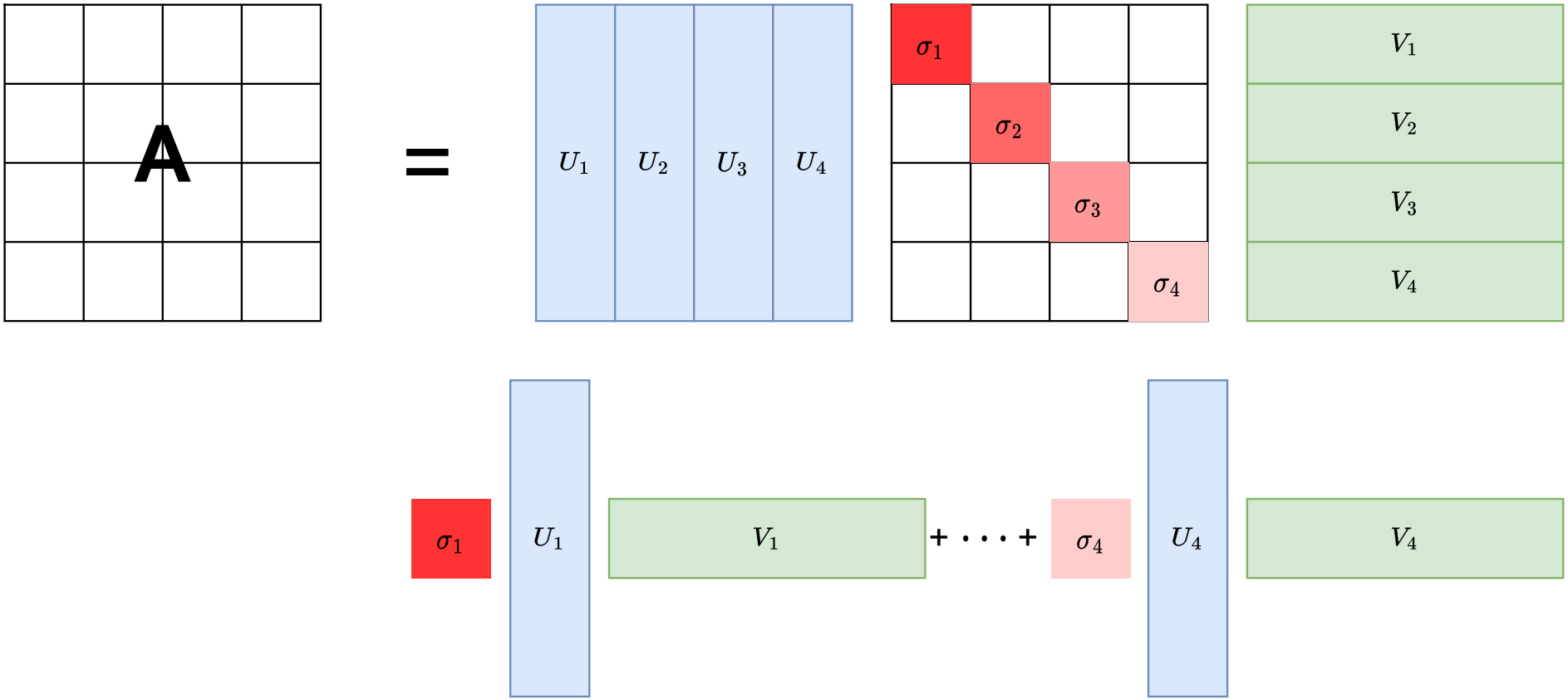
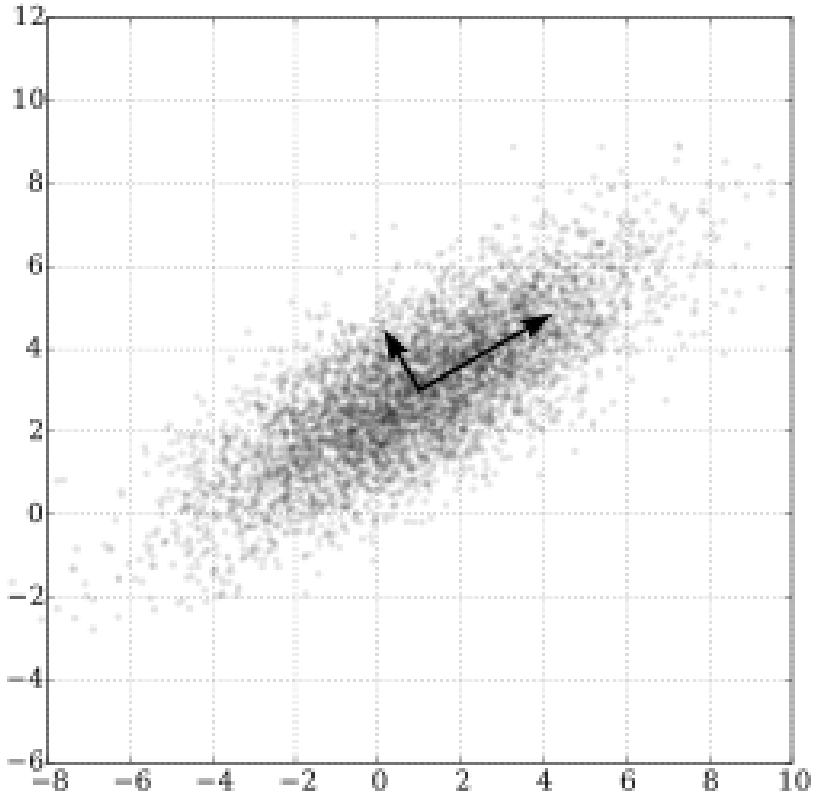


Figura: Fuente:[1]

Reducción de Dimensionalidad III



Principal Component Analysis (PCA)

Método *lineal* que busca la reducción de las dimensiones en un conjunto de datos, buscando la maxima **varianza** entre ellos.
Dependen de la *proyección vectorial*.

Es posible llegar desde la SVD a PCA teniendo en cuenta que las matrices deben estar centradas (restar la media).

$$M = U\Sigma V^*$$

$$VM = U\Sigma$$

Referencias


Luis Serrano.
Grokking Machine Learning.
Simon and Schuster, 2021.

`https://www.youtube.com/watch?v=wp3AJ0ZJCjw`